

Movie Ticket Booking System

Code Flow Explained Through Real-Life Scenarios
and Their Fallback Mechanisms

Spring Boot 3.3

Java 21

PostgreSQL

Concurrency-Safe

Technical walkthrough · Version 1.0

Contents

1. System at a glance (architecture & layers)
2. The booking state machine
3. Scenario 1 — Two people grab the last seat at the same instant
4. Scenario 2 — A customer selects seats then walks away (abandoned hold)
5. Scenario 3 — Booking a premium recliner for a Saturday night show (pricing)
6. Scenario 4 — Applying a discount coupon at payment
7. Scenario 5 — Payment succeeds but something fails midway (atomicity)
8. Scenario 6 — Customer cancels the ticket and expects a refund
9. Scenario 7 — Confirmation SMS should not slow down checkout (async)
10. Scenario 8 — Admin sets up a new multiplex and show
11. Scenario 9 — Wrong person tries a privileged action (auth & roles)
12. Fallback mechanisms — consolidated reference

1. System at a glance

The service follows a classic layered architecture. Every write path is transactional, and the `ShowSeat` row is the single point of contention that all concurrency control revolves around.

```
Client (Swagger / app)
|
Controller      thin REST layer, validates DTOs, applies @PreAuthorize
|
Service          @Transactional business logic (BookingService, ShowService, ...)
|
Repository       Spring Data JPA (pessimistic lock query lives here)
|
PostgreSQL       row locks (FOR UPDATE) + partial unique index backstop

Side channels:
- ApplicationEventPublisher -> NotificationListener (AFTER_COMMIT, @Async)
- AuditService -> audit_event (append-only trail)
- HoldExpiryScheduler (@Scheduled) -> HoldExpiryService (releases stale holds)
```

Concern	Where it lives
Seat hold / confirm / cancel	<code>service/BookingService.java</code>
Pessimistic lock query	<code>repository/ShowSeatRepository.lockByIds</code>
Auto-release of expired holds	<code>scheduling/HoldExpiryScheduler</code> + <code>service/HoldExpiryService</code>
Pricing (tier + weekend)	<code>service/PricingService</code>
Discounts	<code>service/DiscountService</code>
Refunds	<code>service/RefundPolicyService</code>
Async notifications	<code>notification/NotificationListener</code>
Errors & fallback responses	<code>common/GlobalExceptionHandler</code> + <code>common/ErrorCode</code>

2. The booking state machine

```
ShowSeat:  AVAILABLE --hold--> HELD --confirm--> BOOKED
            ^                |                |
            |                (expiry/cancel)  (cancel/refund)
            +-----+-----+

Booking:   PENDING --confirm--> CONFIRMED --cancel--> CANCELLED / REFUNDED
            |
            (hold lapses) --> EXPIRED
```

3. Scenario 1 — Two people grab the last seat at the same instant

REAL LIFE

A blockbuster is almost sold out. Seat **A10** is the last good seat. Riya (Mumbai) and Arjun (Pune) both tap "Book A10" within the same 50 milliseconds. Only one ticket exists — the system must pick exactly one winner and tell the other the seat is gone. It must never sell A10 twice.

CODE FLOW

- Both requests hit `POST /shows/{id}/holds` → `BookingController.hold` → `BookingService.hold(...)` (each in its own transaction).
- Each transaction calls `ShowSeatRepository.lockByIds([A10])`, which runs `SELECT ... FOR UPDATE` ordered by id. **Layer 1 (pessimistic lock):** PostgreSQL grants the row lock to one transaction; the other *blocks* and waits.
- The winner sees A10 as `AVAILABLE`, flips it to `HELD`, sets `heldUntil`, creates a `PENDING` booking + an active `booking_seat` row, and commits — releasing the lock.
- The loser now acquires the lock, re-reads A10, and finds it is `HELD`, not `AVAILABLE`.

FALLBACK / SAFETY NET

- **Primary outcome:** the loser is rejected cleanly with `SEAT_UNAVAILABLE` (HTTP 409) — thrown in `BookingService.hold` when status is not `AVAILABLE`.
- **Layer 2 (optimistic @Version):** if two paths somehow modified the same seat, the stale writer gets an `OptimisticLockingFailureException`, mapped to 409 by `GlobalExceptionHandler`.
- **Layer 3 (DB backstop):** even if application logic had a bug, the partial unique index `uq_booking_seat_active` on `booking_seat(show_seat_id) WHERE active` makes a second active allocation physically impossible — the insert fails with a `DataIntegrityViolationException` → 409. The seat cannot be oversold.

4. Scenario 2 — A customer selects seats, then walks away

REAL LIFE

Meera picks 3 seats to lock them while she finds her wallet, then gets distracted and closes the app. Those seats must not stay blocked forever — other customers should get them back automatically after a short window.

CODE FLOW

- Her hold set `heldUntil = now + booking.hold.ttl-seconds` (default 120s) and a `PENDING` booking with `holdExpiresAt`.
- `HoldExpiryScheduler.sweepExpiredHolds()` runs every `booking.hold.sweep-interval-ms`, finds `PENDING` bookings whose `holdExpiresAt` is in the past, and calls `HoldExpiryService.expireBooking(id)`.
- That releases each seat back to `AVAILABLE`, flips the `booking_seat` rows to `active = false`, and sets the booking to `EXPIRED`. An audit event is written.

FALLBACK / SAFETY NET

- **Confirm-time guard:** if Meera returns a second too late and hits `confirm`, `BookingService.confirm` re-checks `holdExpiresAt`; if lapsed it releases the seats, marks the booking `EXPIRED`, and returns `HOLD_EXPIRED` (409) — so she can never pay for a seat the sweeper already freed.
- **Isolated sweeping:** each expiry runs in its own `REQUIRES_NEW` transaction; if one booking fails to expire (e.g., it was just confirmed by the owner), the scheduler logs it and continues with the rest instead of aborting the whole batch.
- **Bounded staleness:** a freed seat reappears within one sweep interval, an intentional, documented trade-off.

5. Scenario 3 — Premium recliner, Saturday night show

REAL LIFE

A recliner seat for a Saturday premiere should cost more than a regular seat on a Tuesday matinee. Pricing must reflect both the seat class and the weekend premium, and be locked in when the show is published.

CODE FLOW

- Admin creates the show via `POST /admin/shows` → `ShowService.createShow`.
- For every physical seat it materializes a `ShowSeat` and computes the price via `PricingService.computeSeatPrice(basePrice, seatClass, startTime)`:
$$\text{price} = \text{basePrice} \times \text{tierMultiplier}(\text{seatClass}) \times (\text{isWeekend} ? \text{weekendMultiplier} : 1)$$
- Tier multipliers come from the `pricing_tier` table (REGULAR 1.0, PREMIUM 1.5, RECLINER 2.0); weekend is derived from the show start time in the configured zone.
- The resolved price is stored on the `ShowSeat`, so a later price change never alters an in-flight booking.

FALLBACK / SAFETY NET

- **Missing tier:** if no pricing tier is configured for a seat class, `PricingService` defaults the multiplier to `1.0` instead of failing.
- **Rounding:** all money is `BigDecimal` with `HALF_UP` scale 2 — no floating-point drift.
- **Price snapshotting:** because price is frozen on the `ShowSeat` at creation, admin edits to base price/tiers cannot retroactively overcharge a held seat.

6. Scenario 4 — Applying a discount coupon at payment

REAL LIFE

Karan enters coupon `SAVE20` (20% off, max ₹100, min order ₹300) at checkout. Sometimes the coupon is expired, inactive, or the order is too small — the app must apply it correctly or explain clearly why it can't.

CODE FLOW

- `confirm` calls `DiscountService.computeDiscount(code, total, now)`.
- Valid path: percent or flat value is computed, capped by `maxDiscount`, and never exceeds the order amount.
$$\text{finalAmount} = \text{total} - \text{discount}$$
 is charged and stored on the booking.

FALLBACK / SAFETY NET

- **No code supplied:** returns a zero discount (booking proceeds at full price) — not an error.
- **Invalid / inactive / expired / not-yet-valid / below minimum:** throws `INVALID_DISCOUNT` (HTTP 400) with a specific message; the confirm transaction rolls back and the hold is untouched, so Karan can retry without the code.
- **Over-generous coupon:** a discount larger than the order is clamped to the order amount, so the final payable is never negative.

7. Scenario 5 — Payment step fails midway (atomicity)

REAL LIFE

During confirmation the seats are marked booked, the payment is recorded, and the booking is finalized. If any of these fails, the customer must not end up with a "booked" seat and no payment, or a charge and no seat.

CODE FLOW

- `BookingService.confirm` is a single `@Transactional` unit: mark seats `BOOKED` → save `Payment(SUCCESS)` → set booking `CONFIRMED` → publish event → write audit.
- All of it commits together or not at all.

FALLBACK / SAFETY NET

- **Rollback:** any exception (validation, discount, DB) rolls the whole transaction back — seats revert to their prior state, no orphan payment, no half-booked seat.
- **Notifications gated on commit:** the confirmation event is delivered with `@TransactionalEventListener(AFTER_COMMIT)`, so if the transaction rolls back, **no** "booking confirmed" message is ever sent.
- **Idempotent payment record:** a unique constraint on `payment.booking_id` prevents duplicate payment rows for one booking.

8. Scenario 6 — Customer cancels and expects a refund

REAL LIFE

Plans change. Sara cancels her confirmed booking. How much she gets back depends on how close to showtime she cancels: full refund if early, partial if closer, nothing if too late.

CODE FLOW

- `POST /bookings/{id}/cancel` → `BookingService.cancel`.
- `RefundPolicyService.computeRefund(finalAmount, showStart, now)` applies the active tiered policy: full refund if cancelled $\geq$ `fullRefundHoursBefore`, partial % if $\geq$ `partialRefundHoursBefore`, else zero.
- Seats are released to `AVAILABLE` (`booking_seat.active = false`), the payment is marked `REFUNDED` / `PARTIALLY_REFUNDED`, and the booking becomes `REFUNDED` (or `CANCELLED` if zero refund).

FALLBACK / SAFETY NET

- **Wrong state:** cancelling a booking that is not `CONFIRMED` (e.g., still `PENDING` or already refunded) returns `REFUND_NOT_ALLOWED` (409).
- **Ownership check:** attempting to cancel someone else's booking returns `FORBIDDEN` (403).
- **No policy configured:** `computeRefund` safely returns zero rather than throwing, and the freed seat is immediately re-bookable (the partial unique index permits a new active allocation).

9. Scenario 7 — Confirmation SMS must not slow checkout

REAL LIFE

Sending email/SMS/push can be slow. The customer should get an instant "Booked!" response; notifications happen in the background and must fan out to the right people.

CODE FLOW

- On confirm/cancel, the service publishes a `BookingEvent`.
- `NotificationListener` handles it with `@TransactionalEventListener(AFTER_COMMIT)` and `@Async("notificationExecutor")` — a bounded worker pool (core 4, max 8, queue 500).
- The HTTP response returns as soon as the transaction commits; delivery (mocked with logging here) runs off-thread.

FALLBACK / SAFETY NET

- **Never on rollback:** `AFTER_COMMIT` guarantees no message for a booking that didn't actually happen.
- **Backpressure:** the bounded queue absorbs bursts; the request path is never blocked by notification latency.
- **Non-fatal:** a notification failure cannot roll back or fail the (already committed) booking.

10. Scenario 8 — Admin sets up a new multiplex and show

REAL LIFE

A new PVR opens. An operations admin adds the city, theater, screens, the seat layout, the movie, and finally schedules shows — after which customers can immediately browse and book.

CODE FLOW

- Admin-only endpoints under `/admin/**` (`AdminCatalogController` , `AdminShowController`) create city → theater → screen → seat layout (bulk) → movie → show.
- Creating the show materializes priced `ShowSeat` rows (Scenario 3), which then appear in `GET /shows/{id}/seats` .

FALLBACK / SAFETY NET

- **Validation:** DTO constraints reject blank names, non-positive prices, zero seat counts (`VALIDATION_ERROR` , 400, with field-level detail).
- **Referential integrity:** creating a theater/screen/show with a missing parent id returns `NOT_FOUND` (404); duplicates (e.g., city name) return `CONFLICT` (409).

11. Scenario 9 — Wrong person tries a privileged action

REAL LIFE

A regular customer (or an anonymous user) tries to create a discount code or edit shows. Only admins may do that.

CODE FLOW

- Requests carry a JWT (from `/auth/login`); `JwtAuthenticationFilter` populates the security context.
- Method/endpoint rules (`@PreAuthorize("hasRole('ADMIN')")` , security config) authorize the call.

FALLBACK / SAFETY NET

- **No/expired token:** `UNAUTHORIZED` (401) via the authentication entry point.
- **Valid token, wrong role:** `FORBIDDEN` (403) via the access-denied handler.
- **Ownership:** customers can only act on their own bookings (403 otherwise).

12. Fallback mechanisms — consolidated reference

Situation	Mechanism	Result
Concurrent booking of same seat	Pessimistic <code>FOR UPDATE</code> lock (deterministic order)	Serialized; one winner
Lost update on seat/booking	Optimistic <code>@Version</code>	409 CONFLICT
Logic bug allowing double allocation	Partial unique index on <code>booking_seat</code>	DB rejects; 409
Seat unavailable at hold	Status check in <code>hold</code>	SEAT_UNAVAILABLE (409)
Abandoned hold	<code>@Scheduled</code> expiry sweeper	Seat auto-released; booking EXPIRED
Confirm after hold lapsed	Expiry re-check in <code>confirm</code>	HOLD_EXPIRED (409)
One booking fails during sweep	Per-booking <code>REQUIRES_NEW</code> tx	Batch continues
Invalid / expired / small-order coupon	Validation in <code>DiscountService</code>	INVALID_DISCOUNT (400)
Any failure during confirm	<code>@Transactional</code> rollback	All-or-nothing; no orphan state
Notification while committing	AFTER_COMMIT + <code>@Async</code>	Non-blocking; never on rollback
Cancel non-confirmed booking	State check in <code>cancel</code>	REFUND_NOT_ALLOWED (409)
Acting on another user's booking	Ownership check	FORBIDDEN (403)
Missing pricing tier / refund policy	Safe defaults	Multiplier 1.0 / zero refund
Missing parent entity / duplicate	Lookup + uniqueness checks	NOT_FOUND (404) / CONFLICT (409)
Bad input	Bean Validation + handler	VALIDATION_ERROR (400) with fields
Missing / wrong-role token	JWT filter + authorization	UNAUTHORIZED (401) / FORBIDDEN (403)

All error responses share a consistent JSON shape: `{ timestamp, status, code, message, path, fieldErrors? }`, produced by `GlobalExceptionHandler`. The three-layer concurrency defense is validated by an automated 20-thread race test running against a real PostgreSQL instance.